

Review Intelligence for Hotel Revenue Management

Fine-tuned deep-learning classifiers that turn guest reviews into pricing signals

Group	Student name	Student ID	Role
Group 6	Ou Zhiyong	A0018498R	Data preparation; sentiment model (Notebooks 00, 01)
	Zhang Ludan	A0309855R	Guest segment model; experiments; app (Notebooks 02, 03)

1. Company and Project Links

Item	Details
Company	Oriental Horizon Hotel Business
Company website	https://orientalhorizon.com.cn/
Model URLs (fine-tuned)	https://huggingface.co/ludanzhang54/oh-review-sentiment https://huggingface.co/ludanzhang54/oh-guest-segment
App URL (Streamlit Cloud)	https://oriental-horizon-review.streamlit.app/
GitHub URL (linked to app)	https://github.com/ludanzhang54/MBA/tree/claude/homework-help-8tt9tf/ISOM5240_GroupProject
Prepared dataset	https://huggingface.co/datasets/ludanzhang54/oriental-horizon-hotel-reviews

2. Project Objective

This project uses fine-tuned deep-learning text classifiers to turn Oriental Horizon's guest reviews into two pricing signals for its revenue management system: guest sentiment, which measures reputation against competitors, and guest segment (business or leisure), which guides segment-specific room pricing. (40 words)

3. Strategy

Oriental Horizon prices its rooms with a revenue management system (RMS). The RMS works with numeric inputs such as occupancy, booking pace and competitor rates. Guest reviews also influence what guests are willing to pay, but they arrive as free text, so the RMS cannot use them today. Our strategy converts that text into two numbers the RMS can use:

- **Reputation signal.** A sentiment classifier labels each review as negative, neutral or positive. The share of positive reviews minus the share of negative reviews gives a **Net Reputation Score (NRS)** from -100 to +100. If the hotel's NRS is above the competitor benchmark, the app suggests a small rate premium (+1% per 10 NRS points, capped at $\pm 5\%$); if it is below, it suggests a discount and flags the reviews to fix.
- **Demand-mix signal.** A segment classifier labels each review as written by a **business** or **leisure** guest. Business guests stay on weekdays and are less price-sensitive; leisure guests stay at weekends and in holidays and compare prices. The segment mix tells the revenue manager which pricing levers to use.

We followed five steps:

1. **Data.** Hotel reviews are confidential, so we used a large public set of Booking.com reviews as a stand-in. We derived the labels from information already in the data (reviewer score and trip tags), so no manual labelling was needed.
2. **Fine-tuning.** For each task we fine-tuned two small pre-trained Hugging Face models (DistilBERT and DistilRoBERTa) on class-balanced data and selected one on the validation set.
3. **Experiments.** We compared the fine-tuned models with pre-trained models used without fine-tuning, measuring accuracy and CPU runtime on held-out test sets.

4. **Iteration.** A first version with three guest segments was too inaccurate (55.8%). We re-defined the segments as business vs. leisure and enlarged the training data (Section 7).
5. **Deployment.** A Streamlit app runs both fine-tuned pipelines, turns the predictions into pricing suggestions, and includes a live accuracy check on Streamlit Cloud.

4. Dataset

Source: *515K Hotel Reviews Data in Europe*, roughly 515,000 Booking.com reviews of 1,493 European hotels (public domain),

Kaggle: <https://www.kaggle.com/datasets/jiashenliu/515k-hotel-reviews-data-in-europe>. Prepared, labelled splits: <https://huggingface.co/datasets/ludanzhang54/oriental-horizon-hotel-reviews>.

Pre-processing (Notebook 00):

- Booking.com stores the *liked* and *disliked* parts of a review separately and writes placeholders ("No Positive", "No Negative") when a part is empty. We removed the placeholders and joined both parts into one free-text review, then dropped empty and duplicate reviews.
- **Sentiment label** from the reviewer score: below 5.5 = negative, 6.5–7.9 = neutral, 9.0 and above = positive. Scores in the gaps (5.5–6.4 and 8.0–8.9) were left out because they are ambiguous even for a human reader.
- **Segment label** from the trip tags: "Business trip" = business, "Leisure trip" = leisure. Reviews without a trip tag were left out.
- Minimum length: 5 words (sentiment) and 20 words (segment), because the purpose of a trip is rarely visible in very short reviews.
- **Class balancing:** positive reviews far outnumber negative ones, so we drew the same number of reviews per label (15,000) and split them 80/10/10 into train, validation and test sets, keeping label proportions equal (stratified, random seed 42).

	Sentiment task	Guest segment task
(a) Number of labels	3 — negative, neutral, positive	2 — business, leisure
(b) Relevant features	Review text (input); reviewer score → label	Review text (input); trip tags → label
(c) Training samples	36,000 (12,000 per label)	24,000 (12,000 per label)
Validation samples	4,500 (1,500 per label)	3,000 (1,500 per label)
Testing samples	4,500 (1,500 per label)	3,000 (1,500 per label)
Average review length	41 words	53 words
(d) Sources	Kaggle 515K Hotel Reviews Data in Europe	Same

Table 1. Dataset summary (final version, Notebook 00).

5. Model

The application chains **two Hugging Face text-classification pipelines**, each running a model we fine-tuned for one business question (Figure 1).

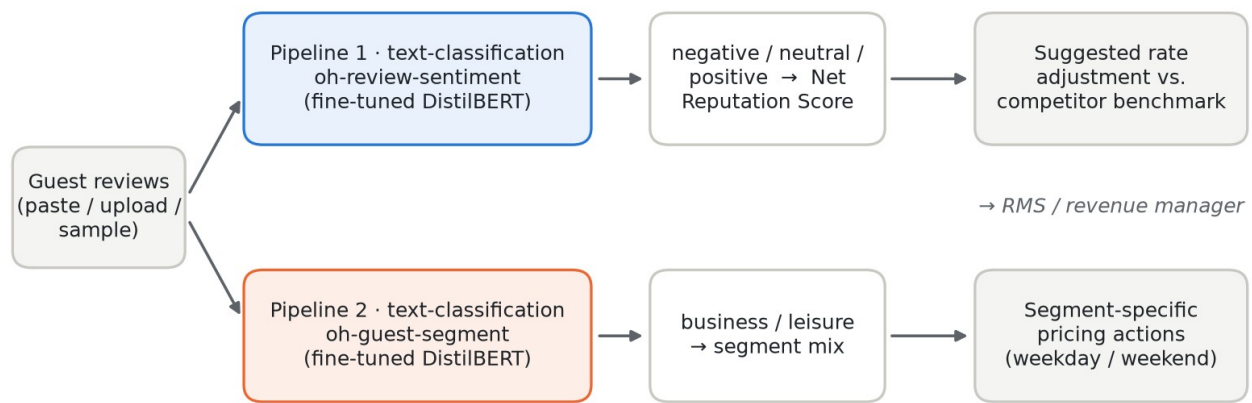


Figure 1. Application pipeline: two fine-tuned classifiers feed the reputation and demand-mix signals.

Role	Hugging Face model	Used for
Pre-trained base (candidate 1)	distilbert-base-uncased (67M parameters)	Fine-tuned for both tasks — selected
Pre-trained base (candidate 2)	distilroberta-base (82M parameters)	Fine-tuned for both tasks — compared
Baseline, no fine-tuning	nlptown/bert-base-multilingual-uncased-sentiment	Sentiment baseline (1–5 stars mapped to 3 classes)
Baseline, no fine-tuning	facebook/bart-large-mnli	Segment baseline (zero-shot classification)
Final model 1	ludanzhang54/oh-review-sentiment	Pipeline 1 in the app
Final model 2	ludanzhang54/oh-guest-segment	Pipeline 2 in the app

Table 2. Pre-trained and fine-tuned models.

Why DistilBERT-size models? Both tasks are sentence-level classification, where encoder models such as BERT are well suited. Distilled versions keep most of BERT’s accuracy with 40% fewer parameters, which matters because the app runs on a CPU on Streamlit Cloud. Large language models were avoided, as the course recommends.

Fine-tuning set-up (Notebooks 01 and 02, Google Colab T4 GPU): a new classification head on top of the pre-trained encoder; maximum length 256 tokens; 2 epochs; learning rate 3e-5 with 10% warm-up and linear decay; batch size 32; weight decay 0.01; mixed precision; random seed 42. The model is evaluated on the validation set after each epoch and the best epoch is kept. **Selection rule:** highest validation macro-F1; if two candidates are within 0.5 percentage points, the smaller model is chosen.

6. Deployment

The app is deployed on Streamlit Community Cloud at <https://oriental-horizon-review.streamlit.app/> and loads both fine-tuned models directly from the Hugging Face Hub. A revenue manager uses it in four steps:

1. **Choose reviews.** Paste reviews (one per line), upload a CSV/TXT file, or draw random reviews from the test set. The sidebar holds the current base rate, the competitor NRS benchmark and the maximum allowed adjustment.
2. **Analyse.** Both pipelines classify every review. The app shows the number of reviews, the Net Reputation Score versus competitors, the suggested rate change and the new base rate, plus the sentiment and segment distributions.
3. **Act.** The *Pricing actions* panel explains the suggested adjustment and lists pricing levers for the main guest segments. A table shows the negative share by segment, and every review can be downloaded with its predictions.
4. **Check accuracy.** The *Accuracy check* tab runs both models on labelled test reviews on Streamlit Cloud itself and shows accuracy and the confusion matrix.

Oriental Horizon • Review Intelligence for Revenue Management

Turns guest reviews into reputation and guest-segment signals for room pricing.

 Analyse reviews  Accuracy check  About

Reviews to analyse

☐ Paste reviews ☐ Upload a file ☒ Random test reviews

Number of reviews



 Analyse reviews

Figure 2. Input options: paste reviews, upload a file, or draw random test reviews.

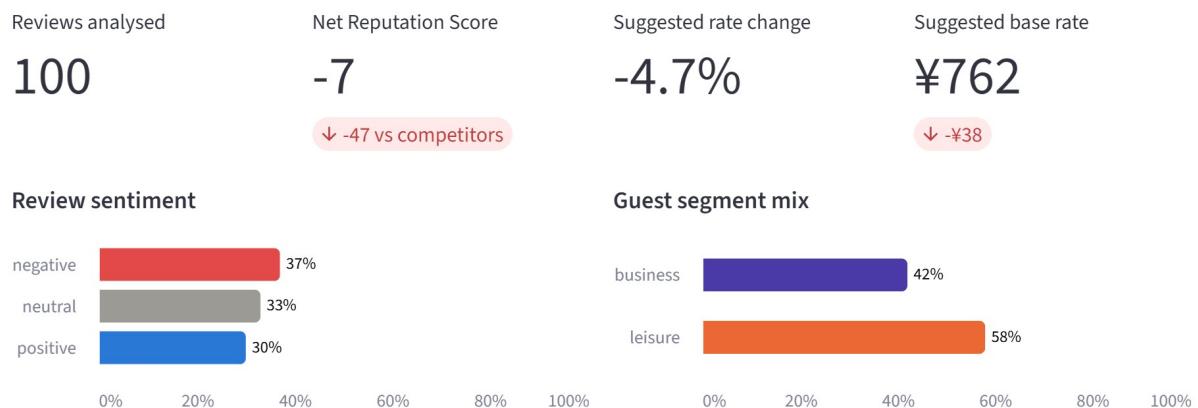


Figure 3. Results: reputation score, suggested rate and the two distributions.

Note: the random test reviews are balanced by design (one third per sentiment label), so their NRS (-7) is far below a real hotel's, and the app correctly suggests a discount. Real review streams are mostly positive.

Pricing actions

Reputation is 47 points below competitors: a premium is hard to justify. Consider -4.7% (¥800 → ¥762) and fix the issues in the negative reviews below.

 **Leisure guests** travel at weekends and in holidays, compare prices and book further ahead. Raise weekend and holiday rates on peak dates, sell packages (breakfast, upgrades, late check-out) instead of discounting, and use early-booking and length-of-stay offers to fill quiet dates. (58% of reviews)

 **Business travellers** book mostly Monday–Thursday and are less price-sensitive. Raise weekday rates on high-demand dates, protect rooms for negotiated corporate rates, and sell late check-out and fast Wi-Fi. (42% of reviews)

Figure 4. Pricing actions generated from the two model outputs.

Sentiment model

Accuracy on 300 test reviews

73.3%

Confusion matrix (rows = true label, columns = prediction)

true label	negative	neutral	positive
negative	71	21	1
neutral	22	63	18
positive	3	15	86

Segment model

Accuracy on 300 test reviews

74.3%

Confusion matrix (rows = true label, columns = prediction)

true label	business	leisure
business	108	32
leisure	45	115

Figure 5. Live accuracy check on Streamlit Cloud (300 test reviews per model).

7. Experiments

(a) Objectives

The experiments had two goals: **(i) select the best model for each task**, judged by accuracy and runtime only; and **(ii) measure the accuracy of the deployed application on Streamlit Cloud**. All tests use held-out test reviews that were never seen during training or model selection. Large test sets (4,500 and 3,000 reviews) keep the standard error of an accuracy figure around 0.7–0.8 percentage points.

(b) Models explored and path to the final pipeline

Iteration 1 used 6,000 reviews per label and three guest segments (business / couple / family). The sentiment model reached 72.7% test accuracy, but the segment model reached only **55.8%**: one family review in three was predicted as *couple*, because most reviews never mention who the guest travelled with.

Iteration 2 made two changes: (1) segments were redefined as **business vs. leisure**, the core segmentation in hotel revenue management and one that reviews reveal more often; (2) the training data were enlarged 2.5 times to 15,000 reviews per label. Sentiment accuracy rose to 75.0%, and the new segment model reached 72.1%.

Iteration	Data per label	Sentiment test accuracy	Segment test accuracy
1	6,000	72.7% (n = 1,800)	55.8% — 3 segments (n = 1,800)
2 (final)	15,000	75.0% (n = 4,500)	72.1% — business vs. leisure (n = 3,000)

Table 3. Model iterations (selected DistilBERT models).

(c) Results

Experiment 1 — model selection (Notebook 03). Runtime is measured on a CPU, one review at a time, which matches how the app runs on Streamlit Cloud.

Task	Model	Approach	Accuracy	Macro-F1	CPU ms / review
Sentiment	nlptown BERT multilingual	Pre-trained, no fine-tuning	62.8%	0.605	149
Sentiment	DistilBERT ✓	Fine-tuned	75.0%	0.750	107
Sentiment	DistilRoBERTa	Fine-tuned	75.9%	0.760	78

Task	Model	Approach	Accuracy	Macro-F1	CPU ms / review
Segment	BART-large-MNLI	Zero-shot, no fine-tuning	55.4%	0.462	1,693
Segment	DistilBERT ✓	Fine-tuned	72.1%	0.721	83
Segment	DistilRoBERTa	Fine-tuned	72.0%	0.720	75

Table 4. Model selection on the full test sets (sentiment $n = 4,500$; segment $n = 3,000$). ✓ = model used in the app.

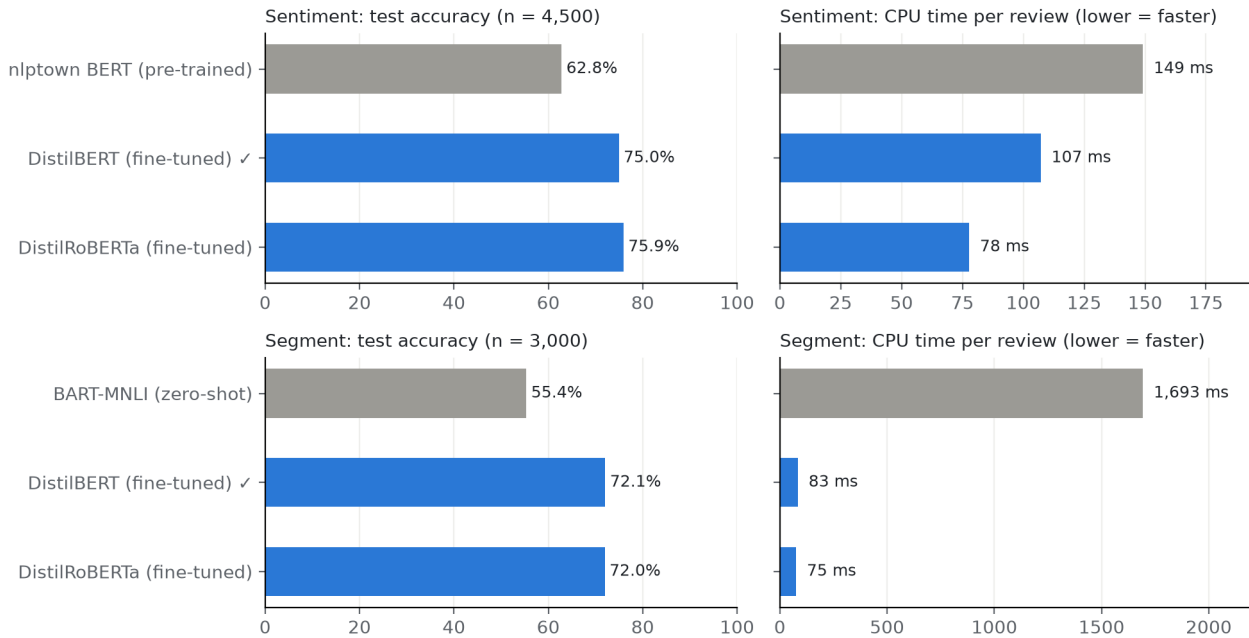


Figure 6. Test accuracy and CPU runtime of all candidate models.

Findings.

- **Fine-tuning adds 12–17 points of accuracy.** The pre-trained sentiment model rarely recognises mixed (neutral) reviews (recall 30% vs. 65% after fine-tuning). The zero-shot BART model labels almost every review as leisure (business recall 14%) and is about 20 times slower, so it is unsuitable for the app.
- **The two fine-tuned candidates are statistically tied** (differences ≤ 0.9 points, about 1.4 standard errors). Our pre-set rule therefore chose the smaller DistilBERT. The runtime test later showed DistilRoBERTa is slightly faster on CPU, so parameter count was a poor proxy for speed; measured latency is the better tie-breaker for future versions.
- **Error pattern (Figure 7).** Almost all sentiment errors are one step away (neutral vs. positive or negative). Positive and negative are confused in only 42 of 3,000 reviews (1.4%), which keeps the Net Reputation Score reliable. Segment errors are balanced in both directions (432 vs. 406), so the model does not favour either segment.

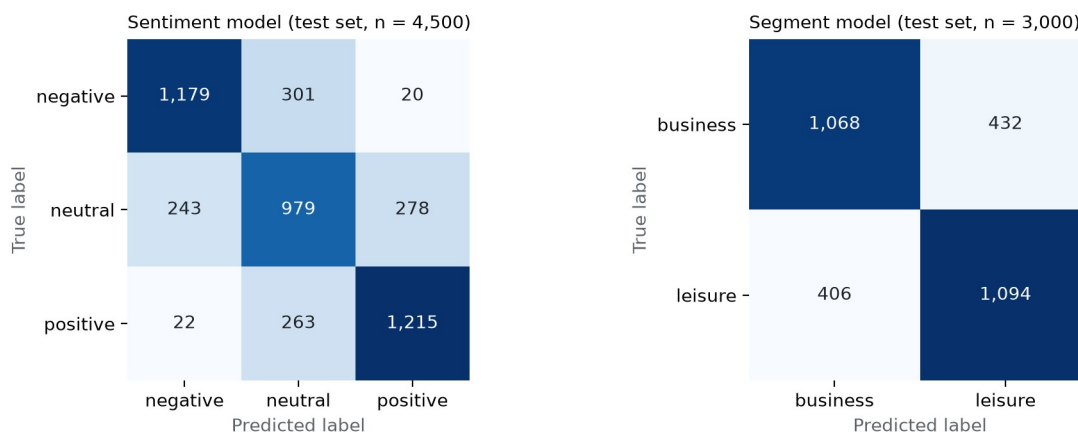


Figure 7. Confusion matrices of the two final models on the test sets.

Model	Label	Precision	Recall	F1-score
Sentiment (DistilBERT)	negative	0.817	0.786	0.801
	neutral	0.635	0.653	0.643
	positive	0.803	0.810	0.807
Segment (DistilBERT)	business	0.725	0.712	0.718
	leisure	0.717	0.729	0.723

Table 5. Per-class results of the final models (full test sets).

Experiment 2 — application performance on Streamlit Cloud. The *Accuracy check* tab ran both deployed models on 300 random test reviews each (seed 42). Notebook 03 repeated the identical test in Colab. The results match exactly, which confirms that the app uses the same models the notebooks produced.

Pipeline in the app	Test reviews	Accuracy on Streamlit Cloud	Same test in Notebook 03
Sentiment (oh-review-sentiment)	300	73.3%	73.3%
Guest segment (oh-guest-segment)	300	74.3%	74.3%

Table 6. Application accuracy. Both figures lie within the sampling margin (about ± 5 points at $n = 300$) of the full-test-set results.

8. Conclusion

(a) Was the business problem addressed?

Largely, yes. The sentiment pipeline classifies reviews with 75% accuracy and almost never mixes up positive and negative feedback, so the Net Reputation Score built from it is a dependable reputation signal for pricing. The segment pipeline reaches 72% on a balanced business-vs-leisure test. That is reliable enough to estimate the segment mix across many reviews, but not to label an individual guest. Both fine-tuned models clearly beat pre-trained alternatives in accuracy and speed, and the deployed app returns pricing suggestions for 100 reviews in seconds.

Limitations. (1) The models were trained on English reviews of European hotels as a stand-in for Oriental Horizon's own reviews. (2) Labels come from scores and tags, which are noisy: a guest may give 9.0 to a review full of complaints. (3) With 72% per-review accuracy, the estimated segment shares are pulled towards 50/50; they can be corrected using the known confusion matrix. (4) The pricing rule (+1% per 10 NRS points, capped at $\pm 5\%$) is a transparent starting point that should be calibrated against booking data.

(b) How to implement the application

1. **Collect reviews weekly** from Oriental Horizon's booking channels (OTAs, website, surveys) and the same for 3–5 competitor hotels, and run both pipelines in batch.
2. **Feed the RMS two indicators:** the hotel's NRS relative to the competitor average (a reputation index), and the business/leisure mix by month and day of week (a demand-mix input).
3. **Keep the revenue manager in control:** suggestions stay within the agreed cap and are approved before rates change; the *negative share by segment* view goes to operations to fix recurring problems.
4. **Retrain on Oriental Horizon's own data:** label a sample of the hotel's reviews (many in Chinese) and fine-tune a multilingual or Chinese model (e.g. xlm-roberta-base or bert-base-chinese) with the same notebooks.
5. **Monitor:** repeat the accuracy check every quarter on newly labelled reviews and compare suggested rate changes with actual revenue per available room (RevPAR).